Republic of the Philippines

North Eastern Mindanao State University

ByteQuest: A Gamified Programming Learning Platform Using Comparative Machine Learning Models for Predicting Programming-Task Performance

An Undergraduate Thesis
Presented to the faculty of College of Information Technology Education
NORTH EASTERN MINDANAO STATE UNIVERSITY
Tandag City, Surigao Del Sur

In Partial Fulfillment
of the Requirements for the Degree of
Bachelor of Science in Computer Science

By:

Dagohoy, Venz Andrew P.
Loren, Leandro Kieser Y.
Mangin, Eliamen A.
Somogba, Mares Susette V.

October 2026

Tandag, Surigao del Sur 8300
086-214-4221
records@nemsu.edu.ph

CHAPTER II

REVIEW OF RELATED LITERATURE AND STUDIES

Related Literature

Foreign

Costa (2023) examined the effects of game concepts on programming learning through a multi-level meta-analysis. The study analyzed the effects of commonly used gamification elements, including points, badges, levels, avatars, and leaderboards. The findings showed that gamification produced a significant positive effect on programming learning, although the overall effect was primarily associated with the use of levels. Points, badges, avatars, and leaderboards did not demonstrate significant effects on programming learning outcomes, although these elements may still contribute to student motivation and engagement. These findings provide a basis for incorporating structured progression into ByteQuest while treating other game elements primarily as engagement mechanisms rather than assuming that every game mechanic directly improves academic performance.

Similarly focusing on game-based learning, Maryono et al. (2022) reviewed

the implementation of gamification in programming learning through a systematic literature review. The review examined the difficulties students experience when learning programming, the technologies and game features used in programming-learning environments, and the effects of gamification on programming learning. The authors reported that gamification can increase students' interest, engagement, and motivation, while also identifying limitations related to implementation, technology, and variations in learning outcomes. This literature provides a foundation for using game-based elements in ByteQuest to encourage continued programming practice. It also indicates that gamification should be aligned with instructional objectives rather than being treated as an independent solution to programming-learning difficulties.

Transitioning from engagement strategies to technical evaluation, Messer et al. (2024) conducted a systematic review of automated grading and feedback tools for programming education, analyzing 121 research papers. The review found that automated assessment systems commonly use dynamic techniques such as unit testing to determine the correctness of programming assignments, while other systems employ static analysis to examine source code. Automated assessment can provide near-instantaneous feedback and allow multiple submissions, which may increase opportunities for students to practice and improve. However, the authors also noted limitations in the types of feedback provided and challenges in

comparing systems because assessment datasets are not always publicly available. These findings support ByteQuest's use of automated assessment as a mechanism for evaluating submitted programs while recognizing that assessment results should be distinguished from predictive modeling.

Alongside automated evaluation systems, the integration of artificial intelligence presents another significant pedagogical advancement. Specifically, Prather et al. (2023) examined the emerging role of generative artificial intelligence and large language models in computing education. Their working-group report synthesized findings from 71 primary articles, surveyed computing students and instructors from 20 countries, interviewed computing educators, and considered ethical issues associated with LLM use. The report identified opportunities for LLMs to support programming instruction while also emphasizing concerns related to assessment, academic integrity, reliability, and responsible implementation. The literature provides a basis for considering LLMs as a supporting technology in ByteQuest for generating learning materials and programming activities. However, the findings also indicate that generated content should be evaluated rather than assumed to be accurate or pedagogically appropriate.

Expanding on the practical and ethical challenges of this technology**,** Yan et al. (2024) conducted a systematic scoping review of the issues associated with

large language models in education. The review examined a broad body of educational LLM research and identified issues related to technological readiness, transparency, replicability, privacy, and human-centered implementation. The findings indicate that the educational use of LLMs requires careful consideration of how generated information is produced and evaluated. This literature is relevant to ByteQuest's use of Retrieval-Augmented Generation because document grounding can be considered a strategy for providing the language model with relevant instructional context. Nevertheless, RAG should be treated as a grounding mechanism rather than a guarantee that generated content will always be accurate, complete, or free from unsupported information.

Local

Shifting to the local setting, Zamora, Bautista, and Carpio (2024) reviewed the implementation of coding programs within the Philippine school context. Their literature review examined how coding programs can contribute to students' skill development, logical reasoning, creativity, and cognitive development while also considering challenges in curriculum integration and educational technology. The review further discussed approaches for improving coding programs and emphasized the relevance of programming skills to students' academic development and future opportunities. This literature provides a Philippine context

for ByteQuest by highlighting the importance of structured programming instruction and appropriate educational technologies. Although the review focuses on Philippine grade-school coding programs rather than university programming courses, its discussion of curriculum integration and skill development is relevant to the development of a programming-learning environment.

To enhance student engagement within these local educational programs, Guisadio et al. (2025) conducted a systematic review of gamified elements used in Philippine education. The review examined peer-reviewed studies involving game elements such as points, badges, leaderboards, levels, and quests. The authors reported that gamification has been associated with increased motivation, engagement, and learning gains when game elements are appropriately connected with instructional objectives. The review also identified implementation challenges, including unequal access to technology, teacher preparation, and differences in implementation contexts. These findings are relevant to ByteQuest because they provide local evidence that gamification can be integrated into educational environments, while also emphasizing that game mechanics should support clearly defined learning objectives rather than simply provide entertainment.

Further supporting the benefits of game-based learning in problem-solving

disciplines, Nob, Roble, and Lomibao (2024) reviewed research on gamification in the Philippine mathematics-education context. Their systematic review analyzed 31 selected studies and found positive outcomes involving student engagement, motivation, academic performance, and higher-order thinking. The review identified competitive learning environments, collaboration, rewards, feedback mechanisms, and interactive features as factors associated with positive outcomes. However, the authors also reported limited effects in areas such as attitudes, anxiety, and self-efficacy. Although the review focuses on mathematics rather than programming, its findings are relevant to ByteQuest because programming also involves sustained problem solving and repeated practice. The study therefore provides local literature supporting carefully designed gamified learning activities while recognizing that effects can vary according to implementation and learning objectives.

Alongside gamification, the emergence of artificial intelligence represents another critical technological advancement for local educational tools. Addressing this shift, Estrellado and Miranda (2023) reviewed the emerging role of artificial intelligence in the Philippine educational context. The authors discussed opportunities for AI-supported learning and data-driven decision-making while identifying challenges related to technological infrastructure, computing resources, data privacy, the digital divide, and faculty training. The review emphasizes that

successful educational AI implementation requires not only technological capability but also appropriate institutional and ethical considerations. This literature is relevant to ByteQuest because the platform incorporates an AI-assisted component for generating educational materials. It also supports the need to consider responsible implementation, appropriate content review, and the limitations of AI-generated educational resources.

Expanding on the practical readiness for these AI applications, Co (2025) presented a narrative review of artificial intelligence in Philippine education by synthesizing 43 published studies concerning AI applications, stakeholder perceptions, readiness, and implementation challenges. The review reported disparities in AI implementation across Philippine educational settings and emphasized the importance of infrastructure, institutional readiness, technical skills, and appropriate educational policies. These findings provide a local contextual basis for incorporating AI into an educational platform such as ByteQuest. They also reinforce the importance of treating AI as a supporting instructional technology rather than as a replacement for instructional design, assessment procedures, or human evaluation.

Related Studies

Foreign

Transitioning to the platform's predictive modeling component, Rimal et al. (2024) investigated the classification of student results using Random Forest and examined the effect of hyperparameter tuning on model performance. The study compared different tuning approaches and demonstrated that model performance can change substantially depending on the configuration of the learning algorithm. The findings are relevant to ByteQuest because they demonstrate the importance of model configuration when applying Random Forest to educational data. Rather than assuming that Random Forest will automatically produce the best results, ByteQuest will evaluate its performance empirically using the study's own dataset and evaluation criteria. The study therefore provides methodological support for including hyperparameter tuning and systematic model evaluation in the machine-learning component of the platform.

Alongside predictive algorithms, the use of generative artificial intelligence for formative student support represents another key technical feature. Koutcheme et al. (2024) investigated the ability of open-source language models to provide feedback to students in programming courses. The researchers used GPT-4 as an automated judge to evaluate feedback produced by several open-source models and compared the evaluations with human judgments. The study found that GPT-4 showed a tendency to rate feedback positively and achieved only moderate

agreement with human raters, while some open-source models demonstrated competitive performance. These findings demonstrate the potential of LLMs for programming feedback but also emphasize the need for evaluation and quality control. For ByteQuest, this supports using AI-generated explanations as supplementary feedback while maintaining the predefined automated assessment results as the primary basis for determining whether a programming task satisfies its requirements.

Building upon this potential for automated feedback, Leinonen et al. (2023) investigated the use of large language models to improve programming error messages for novice programmers. The study explored how LLMs could transform traditional error messages into explanations and suggestions that are easier for learners to understand and act upon. The results indicated that LLM-generated enhancements can make programming errors more interpretable and actionable in several cases. This study is relevant to ByteQuest because students who fail programming challenges may benefit from explanations that help them understand the nature of their errors. The findings support the use of LLM-generated explanatory feedback as a supplementary layer alongside automated test-case assessment rather than as a replacement for objective program evaluation.

Beyond evaluating student code, language models also demonstrate

significant utility in the direct creation of learning materials. Sarsa et al. (2022) investigated the automatic generation of programming exercises and code explanations using OpenAI Codex. The researchers generated programming exercises, sample solutions, test cases, and code explanations and evaluated the resulting materials qualitatively and quantitatively. The majority of the generated content was considered novel and sensible, and the researchers found that prompts could influence both the programming concepts and contextual themes of generated exercises. However, the study also emphasized the need for oversight to ensure the quality of generated educational materials. This provides direct support for ByteQuest's AI-assisted generation of programming activities while reinforcing the need to review generated content before it is presented to students.

Finally, synthesizing these technical elements within an engaging learning environment requires a strong motivational structure. Addressing this, Mellado et al. (2024) examined the use of gamification in programming courses and investigated its relationship with learning outcomes and gender differences. The study involved university students participating in a programming course and evaluated the effects of gamified learning activities on educational outcomes and student experiences. The findings provide empirical evidence that gamification can be incorporated into programming education and can influence student learning experiences. This study is relevant to ByteQuest because the platform similarly

uses game-oriented mechanisms to support programming practice. However, the present study differs by combining gamification with automated programming assessment and comparative machine-learning prediction of subsequent programming-task performance.

Local

Transitioning to the platform's predictive modeling component, Rimal et al. (2024) investigated the classification of student results using Random Forest and examined the effect of hyperparameter tuning on model performance. The study compared different tuning approaches and demonstrated that model performance can change substantially depending on the configuration of the learning algorithm. The findings are relevant to ByteQuest because they demonstrate the importance of model configuration when applying Random Forest to educational data. Rather than assuming that Random Forest will automatically produce the best results, ByteQuest will evaluate its performance empirically using the study's own dataset and evaluation criteria. The study therefore provides methodological support for including hyperparameter tuning and systematic model evaluation in the machine-learning component of the platform.

Alongside predictive algorithms, the use of generative artificial intelligence

for formative student support represents another key technical feature. Koutcheme et al. (2024) investigated the ability of open-source language models to provide feedback to students in programming courses. The researchers used GPT-4 as an automated judge to evaluate feedback produced by several open-source models and compared the evaluations with human judgments. The study found that GPT-4 showed a tendency to rate feedback positively and achieved only moderate agreement with human raters, while some open-source models demonstrated competitive performance. These findings demonstrate the potential of LLMs for programming feedback but also emphasize the need for evaluation and quality control. For ByteQuest, this supports using AI-generated explanations as supplementary feedback while maintaining the predefined automated assessment results as the primary basis for determining whether a programming task satisfies its requirements.

Building upon this potential for automated feedback, Leinonen et al. (2023) investigated the use of large language models to improve programming error messages for novice programmers. The study explored how LLMs could transform traditional error messages into explanations and suggestions that are easier for learners to understand and act upon. The results indicated that LLM-generated enhancements can make programming errors more interpretable and actionable in several cases. This study is relevant to ByteQuest because students who fail

programming challenges may benefit from explanations that help them understand the nature of their errors. The findings support the use of LLM-generated explanatory feedback as a supplementary layer alongside automated test-case assessment rather than as a replacement for objective program evaluation.

Beyond evaluating student code, language models also demonstrate significant utility in the direct creation of learning materials. Sarsa et al. (2022) investigated the automatic generation of programming exercises and code explanations using OpenAI Codex. The researchers generated programming exercises, sample solutions, test cases, and code explanations and evaluated the resulting materials qualitatively and quantitatively. The majority of the generated content was considered novel and sensible, and the researchers found that prompts could influence both the programming concepts and contextual themes of generated exercises. However, the study also emphasized the need for oversight to ensure the quality of generated educational materials. This provides direct support for ByteQuest's AI-assisted generation of programming activities while reinforcing the need to review generated content before it is presented to students.

Finally, synthesizing these technical elements within an engaging learning environment requires a strong motivational structure. Addressing this, Mellado et al. (2024) examined the use of gamification in programming courses and

investigated its relationship with learning outcomes and gender differences. The study involved university students participating in a programming course and evaluated the effects of gamified learning activities on educational outcomes and student experiences. The findings provide empirical evidence that gamification can be incorporated into programming education and can influence student learning experiences. This study is relevant to ByteQuest because the platform similarly uses game-oriented mechanisms to support programming practice. However, the present study differs by combining gamification with automated programming assessment and comparative machine-learning prediction of subsequent programming-task performance.

Synthesis

The reviewed literature and studies provide several foundations for the development of ByteQuest as a gamified programming-learning platform with predictive analytics. Literature on programming education indicates that programming requires structured learning activities that develop problem-solving, logical reasoning, and practical coding skills. Philippine literature further emphasizes the importance of appropriately designed coding programs and educational technologies in supporting these skills. These findings provide the instructional foundation for ByteQuest's programming activities.

The literature on gamification indicates that game-based mechanisms can support engagement and motivation in educational settings, including programming education. Costa (2023) provides evidence that gamification can improve programming-learning outcomes, although the strongest learning-outcome evidence was associated with level-based progression rather than all individual game mechanics. Maryono et al. (2022) similarly identified positive effects of gamification on interest, engagement, and motivation while recognizing implementation limitations. Philippine reviews by Guisadio et al. (2025) and Nob et al. (2024) further indicate that gamification can produce positive educational outcomes when game elements are appropriately aligned with instructional objectives. These findings support ByteQuest's use of levels, points, challenges, and other game elements while emphasizing that such features should reinforce learning rather than function only as rewards.

The literature on automated programming assessment establishes a separate but important component of the platform. Messer et al. (2024) found that automated programming assessment commonly uses techniques such as unit testing to determine whether submitted programs satisfy specified requirements. Such systems can provide immediate results and support repeated practice. This is consistent with ByteQuest's use of predefined test cases to determine

programming-task outcomes. However, automated assessment and machine-learning prediction serve different purposes. Automated assessment determines the observed outcome of a submitted program, whereas the machine-learning component uses previously observed student-performance features to predict the outcome of a subsequent programming task.

Research on artificial intelligence and large language models provides a basis for ByteQuest's AI-assisted content-generation component. Prather et al. (2023), Yan et al. (2024), Sarsa et al. (2022), Koutcheme et al. (2024), and Leinonen et al. (2023) demonstrate that LLMs can support programming education through exercise generation, explanations, feedback, and error-message enhancement. At the same time, these studies identify limitations involving accuracy, evaluation, ethical concerns, and the possibility of misleading generated content. Philippine literature likewise emphasizes infrastructure, readiness, privacy, and responsible implementation of AI in education. Consequently, ByteQuest uses the LLM as a supporting content-generation component rather than as the primary predictor of student performance. Retrieval-Augmented Generation provides document-grounding context for generating materials from instructor-provided programming resources, while generated materials remain subject to appropriate review.

The strongest methodological foundation for the predictive component comes from educational machine-learning studies. Rimal et al. (2024) demonstrate the importance of hyperparameter tuning and model configuration in educational classification. Olipas (2026), Bernardo et al. (2021), and Bejar (2024) demonstrate that machine-learning classification can be applied to Philippine educational datasets. Yu et al. (2025) is particularly relevant because it compares Logistic Regression and Random Forest in a programming-related educational context. However, existing studies examine different target variables, such as employability, reading proficiency, academic pathways, programming anxiety, or general student results. They do not directly address the prediction of whether a student will successfully complete a subsequent programming task using performance data collected from an integrated gamified programming-learning platform.

Taken together, the reviewed literature and studies indicate that gamification, automated programming assessment, artificial intelligence, and educational machine learning each have established applications in education. However, these components are generally examined separately or with different predictive objectives. The present study addresses this gap by developing ByteQuest as an integrated programming-learning platform that combines gamified programming activities, automated test-case assessment, AI-assisted document-grounded content generation, performance-data collection, and comparative

machine-learning prediction. Specifically, the study compares Logistic Regression and Random Forest to determine their relative performance in predicting the outcome of a subsequent programming task. The comparison is therefore treated as an empirical investigation rather than assuming in advance that one algorithm will outperform the other.

REFERENCES

Bejar, R. E., Jr. (2024). Precision in progress: Leveraging data mining technique to empower career path selection for incoming senior high school students. *International Journal of Research and Innovation in Social Science*, *8*(1), 178–191. https://rsisinternational.org/journals/ijriss/articles/precision-in-progress-leveraging-data-mining-technique-to-empower-career-path-selection-for-incoming-senior-high-school-students

Bernardo, A. B. I., Cordel II, M. O., Lucas, R. I. G., Teves, J. M. M., Yap, S. A., & Chua, U. C. (2021). Using machine learning approaches to explore non-cognitive variables influencing reading proficiency in English among Filipino learners. *Education Sciences*, *11*(10), 628. https://doi.org/10.3390/educsci11100628

Co, S. J. (2025). Artificial intelligence in Philippine education: A narrative review of applications, perceptions, and challenges. *International Research Journal of Science, Technology, Education and Management*, *5*(2), 25–38. https://doi.org/10.5281/zenodo.15902950

Costa, J. M. (2023). Using game concepts to improve programming learning: A multi-level meta-analysis. *Computer Applications in Engineering Education*, *31*(4), 1098–1110. https://doi.org/10.1002/cae.22630

Estrellado, C. J. P., & Miranda, J. C. (2023). Artificial intelligence in the Philippine educational context: Circumspection and future inquiries. *International Journal of Scientific and Research Publications*, *13*(5). https://www.ijsrp.org/research-paper-0523.php?rp=P13712836

Guisadio, R. G., Tordos, A. J., Pocdol, M., Taboada, F. L., Petin, Y. K., Asentista, L., Sumampong, L. J., Catacutan, F., Catamco, E. G., Delatado, K., Lerio, A., Villanueva, M. L., & Cabello, C. (2025). Gamified elements in Philippine education: A systematic review. *Psychology and Education: A Multidisciplinary Journal*, *44*(1), 142–152. https://doi.org/10.70838/pemj.440109

Juguilon, J. R., Ramos, S., Oris, J., Saringan, N. A., & Tamayo, J. (2025). CODE QUEST: A gamification platform for aspiring Java programmers. *The International Journal of Science, Technology and Engineering*, *1*(1). https://ijste.psu.edu.ph/index.php/about/article/view/8

Koutcheme, C., Dainese, N., Sarsa, S., Hellas, A., Leinonen, J., & Denny, P. (2024). Open source language models can provide feedback: Evaluating LLMs' ability to help students using GPT-4-as-a-judge. In *Proceedings of the 2024 ACM Conference on Innovation and Technology in Computer Science Education* (pp. 52–58). ACM. https://dl.acm.org/

Leinonen, J., Hellas, A., Sarsa, S., Reeves, B., Denny, P., Prather, J., & Becker, B. A. (2023). Using large language models to enhance programming error

messages. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education* (pp. 563–569). ACM. https://dl.acm.org/

Maryono, D., Budiyono, Sajidan, & Akhyar, M. (2022). Implementation of gamification in programming learning: Literature review. *International Journal of Information and Education Technology*, *12*(12), 1448–1457. https://doi.org/10.18178/ijiet.2022.12.12.1771

Mellado, R., Cubillos, C., Vicari, R. M., & Gasca-Hurtado, G. (2024). Leveraging gamification in ICT education: Examining gender differences and learning outcomes in programming courses. *Applied Sciences*, *14*(17), 7933. https://doi.org/10.3390/app14177933

Messer, M., Brown, N. C. C., Kölling, M., & Shi, M. (2024). Automated grading and feedback tools for programming education: A systematic review. *ACM Transactions on Computing Education*, *24*(1), 1–43. https://dl.acm.org/

Nob, J. M. R. O., Roble, D. B., & Lomibao, L. S. (2024). Unveiling the effects of gamification on math learning: A literature review in the Philippine context. *Journal of Innovations in Teaching and Learning*, *4*(1), 13–19. https://doi.org/10.12691/jitl-4-1-3

Olipas, C. N. P. (2026). Predictive modeling and explainability of student employability in the Philippines using Random Forest and Shapley Additive Explanations. *Interdisciplinary Journal of Information, Knowledge, and Management*, *21*. https://doi.org/10.28945/5690

Prather, J., Denny, P., Leinonen, J., Becker, B. A., Albluwi, I., Craig, M., Keuning, H., Kiesler, N., Kohn, T., Luxton-Reilly, A., MacNeil, S., Peterson, A., Pettit, R., Reeves, B. N., & Savelka, J. (2023). The robots are here: Navigating the generative AI revolution in computing education. In *Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education*. ACM. https://dl.acm.org/

Rimal, Y., et al. (2024). The accuracy of machine learning models relies on hyperparameter tuning: Student result classification using Random Forest, randomized search, grid search, Bayesian, genetic, and Optuna algorithms. *Multimedia Tools and Applications*, *83*(30), 74349–74364. https://link.springer.com/

Sarsa, S., Denny, P., Hellas, A., & Leinonen, J. (2022). Automatic generation of programming exercises and code explanations using large language models. In *Proceedings of the 2022 ACM Conference on International Computing Education Research* (pp. 27–43). ACM. https://dl.acm.org/

Yan, L., et al. (2024). Practical and ethical challenges of large language models in education: A systematic scoping review. *British Journal of Educational Technology*, *55*(1), 90–112. https://onlinelibrary.wiley.com/

Yu, E. R. II, Pineda, E. D., Tano, I. M., Lagman, A. C., & Victoriano, J. M. (2025). Comparative analysis of supervised machine learning algorithms for predicting student programming anxiety levels. *Journal of Artificial*

Intelligence, Machine Learning and Neural Network, *5*(1), 28–39. https://doi.org/10.55529/jaimlnn.51.28.39

Zamora, P. A., Bautista, N., & Carpio, J. O. (2024). Reengineering of student-teacher coding program for Philippine grade schools curriculum: A literature review. *JPAIR Multidisciplinary Research*, *57*(1), 141–149. https://doi.org/10.7719/jpair.v57i1.897